

1. 단위테스트

📄 0. 읽기 전에..



사실 그동안 프로젝트를 진행해 오면서 **테스트 코드**를 제대로 작성해본 적이 없다.

그냥 단순하게 이렇게 하면 되겠지라고 생각하였으며, 심지어 그게 4학년 1학기까지 이어져 왔다.

하지만 공부를 계속하게 되면서 지식도 늘어나게 되었으며, 프로젝트의 규모도 자연스럽게 커져갔다.

특히 **캡스톤 디자인**에서 테스트 코드를 작성하지 않고 이어나가니 단위 테스트를 작성하지 않았을 때의 단점이 너무나도 뼈저리게 느껴졌다.

곧 졸업작품 마무리가 끝나고 나의 마지막 프로젝트로 2개 정도 개발을 해보려고 한다.

잠깐 테스트 코드를 참고하여 작성해보았는데, 잠깐 적었음에도 불구하고 뭔가 **믿음이 생기기 시작했다**.

또한 매번 서버를 켜다 켜다 하면서 테스트를 실행하는데, 지금 되돌아보니 매우 시간 낭비였다.

테스트 코드를 통해 검증하면 빠른 개발을 할 수 있지 않았을까 하면서 한심함이 느껴졌다.

아직 늦지 않았다. 지금부터라도 테스트 코드 작성에 대해 공부하며 습관으로 만들어 보려고 한다.

참고 자료

<https://plusb3b.kr/pdf-download-page/index.html>

위 주소는 내가 직접 필기한 것을 바탕으로 작성한 내용이다.

보기에는 어렵지만 그래도 도움이 된다면 다운로드를 추천한다.

1. 단위 테스트를 작성해야 하는 이유

코드를 작성할 때마다 시스템에 추가되는 작은 변화를 검증하는 **단위 테스트**를 작성하게 된다.

그다음 모든 단위 테스트를 실행한다. 만약 문제가 있다면 즉시 중단하고 고치게 되며, **문제 발견을 하기가 쉽다.**

테스트들이 쌓이면 **회귀 테스트**를 할 수 있으며, 기존 동작을 깨트리지 않으면서 새로운 변경점을 검증하려고 시간을 많이 소모할 필요가 없다.

단위 테스트를 작성해야 하는 이유는 간단하게 다음과 같이 **4가지**를 말할 수 있다.

1) 코드의 신뢰성 보장

- 이 코드가 내가 의도한 대로 동작함을 증명한다.
- 테스트가 없으면 코드 수정 시 제대로 동작하는지 의문이 생긴다.
- 테스트가 있으면 즉시 확인이 가능하다.

2) 리팩토링을 안전하게

- 시간이 지나면 코드는 수정된다.
- 테스트가 없으면 수정된 동작을 보장할 방법이 없다.
- 단위 테스트가 있다면 수정 후 테스트만 돌려도 문제 파악이 가능하다.

3) 문서 역할

- 테스트 코드는 해당 기능의 **사용 예제이자 설명서** 역할이다.
- 협업 시 기능이 어떻게 쓰이는지 궁금할 때 테스트 코드를 읽을 수 있다.

4) 디자인 품질 향상

- 테스트 가능한 코드는 **의존성이 낮으며 응집도가 높은** 구조가 필요하다.
 - 테스트를 염두에 두면 좋은 설계를 하게 되는데, 이때 **SRP, DIP, 인터페이스 기반 설계**를 할 수 있다.
-

단위 테스트를 작성하지 않은 경우는 어떻게 될까?

1) 수정할 때마다 전체 시스템 불안정

- 새로운 기능 추가 및 기존 코드 수정 시 이전 기능이 깨질 위험이 증가한다.
- 의존 관계가 많은 서비스/도메인 구조라면 리스크가 커진다.

2) 디버깅 시간 증가

- 테스트가 없으면 매번 서버를 띄우고 API 호출, DB 상태를 확인해야 한다.
- 단위 테스트를 하게 되면 **빠른 피드백 개발**을 할 수 있다.

3) 신뢰성 저하

- 협업 시 다른 개발자가 만든 코드를 믿을 수 없다.
- 기능 변경 시 테스트로 확인하지 못하니 사람 간의 커뮤니케이션 의존도가 증가하게 된다.

4) 리팩토링이 무서워짐

- 수정하면 뭐가 깨질지 몰라 코드를 그대로 둔다.
- 결국 **기술 부채** + 손대기 어려운 구조로 변하게 된다.

5) 배포 후 장애

- 테스트를 하지 않는다는 것은 **운영 환경에서 테스트한다**는 것이다.

- 결국 장애가 발생하며 긴급 패치를 반복하게 되고, 신뢰도가 떨어진다.

마치며

테스트 코드 작성은 선택이 아닌 **필수**이다.

지금부터라도 단위 테스트를 작성하는 습관을 들여보자.

분명 더 나은 개발자로 성장할 수 있을 것이다.
